

1. Why do developers prefer NumPy to similar tools like Matlab, Yorick?

Developers prefer to use NumPy over other tools like Matlab for the following reasons:

- NumPy is an accessible and open-source library. You can easily access it and use it anywhere.
- It is a high-performing library integrated with multidimensional arrays and matrices.
- Some popular libraries work on top of NumPy

For Stats and ML: SciPy, Scikit-Learn, SpaCy, Statsmodels

Array Manipulation: Dask, PyTorch, TensorFlow

Visualization: Seaborn, Matplotlib, Altair, Bokeh, Plotly

- These libraries permit developers to quickly perform high-performance and complex mathematical, science, or finance calculations.
- As a developer, you can take advantage of the general purpose feature of Python as it makes it easy for you to connect with programming languages like C, C++, or Fortran.

2. Why is NumPy Array good compared to Python Lists?

NumPy is better than Python Lists for two primary reasons:

- NumPy Array is static and has a fixed size while creating codes. In other cases, Python Lists are dynamic and can grow dynamically.
- NumPy Array can perform vectorised operations and other advanced calculations, but Python Lists can't do these even after having a large set of functions.

3. What does bincount() function?

The main job of the bincount() function is to count the number of times a given value appears in the array of integers. You have to take care of one point that the function takes only positive integers or Boolean expressions as an argument. You can not pass negative integers.

Syntax: array_object.bincount() import numpy as np

```
p = [10,20,30]
```

```
q = [5,1,7,1,8,13]
```

```
s = np.array([0, 2, 3, 0, 3, 3, 3, 0, 0, 4, 2, 1, 7, 5])
```

```
p = np.bincount(p)
```

```
q = np.bincount(q)
```

```
print("Occurance of each digit in p:", p) print("\nOccurance of each digit in q:", q)
```

```
print("\nOccurance of each digit in q:", np.bincount(s))
```

Occurrence of each digit in p: [0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0

0 0 0 0 0 0 0 1]

Occurrence of each digit in q: [0 2 0 0 0 1 0 1 1 0 0 0 0 1]
nce

Occurrence of each digit in q: [4 1 2 4 1 1 0 1]
nce

4. Explain the data types supported by NumPy.

<https://numpy.org/doc/stable/user/basics.types.html>

numpy.bool_ : bool numpy.byte : signed char numpy.ubyte : unsigned char numpy.short : short
numpy.ushort : unsigned short numpy.intc : int

numpy.uintc : unsigned int numpy.int_ : long numpy.uint : unsigned long numpy.longlong : long
long

numpy.ulonglong : unsigned long long

numpy.half / numpy.float16 : Half precision float numpy.single : float

numpy.double : double numpy.longdouble : long double numpy.csingle : float complex

numpy.cdouble : double complex

numpy.clongdouble : long double complex

5. How does the flatten function differs from the ravel function?

Flatten function has been used for generating 1D versions of the multi-dimensional array. The ravel function does the same operation, but both have a difference.

The flatten() function always returns a copy. The ravel() function returns a view of the original array in most cases. Although you can't see or analyze it in the shown output, if you

make changes in the array returned by ravel(), it might change the whole data of the array. This does not happen while using the flatten() function.

Additionally, you can use the ravel function with any easily parseable object but the flatten() function is used with true NumPy arrays.

And, last, ravel() is faster than flatten().

Syntax: array object.flatten() and array.object.ravel()

5. Explaining Axis=0 or Axis=1

Follow - <https://www.sharpsightlabs.com/blog/numpy-axes-explained/>

6. What Is The Preferred Way To Check For An Empty (zero Element) Array?

If you are certain a variable is an array, then use the size attribute. If the variable may be a list or other sequence type, use len().

The size attribute is preferable to len for numpy array because:

```
a = np.zeros((1,0)) print(a.size) # Will print 0 print(len(a)) # will print 1
```

numpy.any(array) can also be used for this. If it returns False then the array is empty.

```
import numpy as np a = np.zeros((1,0)) print(a.size) len(list(a))
0
1
np.any(a) False
```

7. How do you calculate the moving average?

Before knowing how to calculate moving average, know what it means. It refers to a series of averages of fixed-size subsets of the total observation sets. You can also call it running average, rolling average, or rolling means.

The number of observations and size of windows are required for this calculation. You can calculate the moving average using several methods.

Using convolve function is the simplest method which is based on discrete convolution. You have to use a method that calculates discrete convolution to get a rolling mean. You can

convolve with a sequence of np.ones of a length equal to the desired sliding window length. Syntax: array object.convolve(data, window size)

Explanation - <https://stackoverflow.com/a/54628145>

```
import numpy as np
```

```
def moving_average(x, wsize):
```

```
    return np.convolve(x, np.ones(wsize), 'valid') / wsize data = np.array([1,3,2,7,4,8,2])
```

```
    print(data)
```

```
    print("\n Moving Average:") print(moving_average(data,1))
```

```
[1 3 2 7 4 8 2]
```

Moving Average:

```
[1. 3. 2. 7. 4. 8. 2.]
```

8. Vectorisation in NumPy.

Follow - <https://www.askpython.com/python-modules/numpy/numpy-vectorization>

9. What does numpy.r_ do (numpy)?

Documentation - https://numpy.org/doc/stable/reference/generated/numpy.r_.html What it does is row-wise merging.

```
V = np.array([1,2,3,4,5,6 ])
Y = np.array([7,8,9,10,11,12]) print(np.r_[V[0],Y[0],V[3:5],Y[1],V[2],Y[-1]])
[ 1 7 4 5 8 3 12]
```

10. Finding local maxima/minima with Numpy in a 1D numpy array

```
[1,2,1,3,2,4,5,3]
```

Local MAXIMA -> [2, 3, 5] AT INDEX [1, 3, 6] - If both neighbor elements are smaller.

Local MINIMA -> [1, 1, 2, 3] AT INDEX [0, 2, 4, 7] - If both neighbor elements are greater.

Approach would be- a = [1,2,1,3,2,4,5,3] Say for any element at index i (other than extreme elements), for being maxima below condition need to be True.

```
a[i-1] < a[i] & a[i] > a[i+1] => a[i] > a[i-1] & a[i] > a[i+1]
```

So if we need to do two comparisons one for element in left and one for element in right.

Here element wise comparison will happen.

```
a[1:] > a[:-1] # [2,1,3,2,4,5,3] > [1,2,1,3,2,4,5] =>
```

```
[True,False,True,False,True,True,False]
```

```
a[:-1] > a[1:] # [1,2,1,3,2,4,5] > [2,1,3,2,4,5,3] =>
```

```
[False,True,False,True,False,False,True]
```

And at extreme we just need to compare with the right element for index-0 and with the left of the last element. So we will add True for index-0 left comparison and True for last index right comparison.

#Adding True for extreme ends. + represents concatenation

```
[True] + a[1:] > a[:-1] => [True] + [True,False,True,False,True,True,False] a[:-1] > a[1:] + [True] =>
[False,True,False,True,False,False,True] + [True]
```

Now taking and of both side comparison

```
[True] + a[1:] > a[:-1] & a[:-1] > a[1:] + [True]
```

```
-> [True] + [True,False,True,False,True,True,False] & [False,True,False,True,False,False,True] +
[True]
```

We will use np.r_ function for concatenating row wise. Result that we will get will be a boolean index for local maxima values. We will use this boolean index to get local maximas.

```
import numpy as np
```

```
arr = np.random.randint(0,100, 10) print(arr)
```

```
bool_index = np.r_[True, arr[1:] > arr[:-1]] & np.r_[arr[:-1] > arr[1:], True]
```

```
print(bool_index) print(arr[bool_index])
```

```
[99 38 78 82 76 63 58 95 3 0]
[ True False False True False False False True False False] [99 82 95]
#For minima we just need to reverse the comparison sign
import numpy as np
arr = np.random.randint(0,100, 10) print(arr)
bool_index = np.r_[True, arr[1:] < arr[:-1]] & np.r_[arr[:-1] < arr[1:], True]
print(bool_index) print(arr[bool_index])
```

```
[97  2  78  1 36 85 29 42  45 76]
[False  True False  True  False False  True False False False]
[ 2 1  29]
```

11.Rotate Array/ Image by 90 deg clockwise `numpy.rot90(array, k)`

k is int specifies no of rotation.

<https://numpy.org/doc/stable/reference/generated/numpy.rot90.html>

```
a = [[1,2, 3],
      [4,  5, 6],
      [7,  8, 9]]
```

Expected Output:

```
[[ 4,  1],
 7,
[8,  5, 2],
[9,  6, 3]]
a = np.array([[1,  3],
               .
               [4,  6],
               .
               [7,  9]])
```

`np.rot90(a, 3)`

```
array  4,  1],
([[7,
[8,  5, 2],
[9,  6, 3]])
```

12. Fredo and Array Update in python

Fredo is assigned a new task today. He is given an array A containing N integers. His task is to update all elements of the array to some minimum value x, such that the sum of this new array is strictly greater than the sum of the initial array.

Note that x should be as minimum as possible such that the sum of the new array is greater than the sum of the initial array.

```
a = [1,2,3,4,5]
```

Result-> 4

An initial sum of an array is $1+2+3+4+5=15$ When we update all elements to 4, the sum of the array becomes 20 which is greater than the initial sum 15 . Note that if we had updated the array elements to 3, then sum will be 15, which is not greater than initial sum 15. So, 4 is the minimum value to which array elements need to be updated.

```
a = np.array([1,2,3,4,5]) x = np.sum(a)//a.size + 1 print(x)
```

4

13. Get products of all element except at index

You have an array of integers, and for each index you want to find the product of every integer except the integer at that index. Write a function that takes an array of integers(1-D) and returns an array of the products.

Note: Think of a solution without using division operation. I don't know how to do that, that's why I'm leaving it up to you.

The array [1, 7, 3, 4] Would return:

[84, 12, 28, 21]

By calculating:

```
[7*3*4, 1*3*4, 1*7*4, 1*7*3]
```

```
#
```

```
arr = np.array([1, 7, 3, 4])
```

```
# Approach with division
```

```
print(np.prod(arr)/arr) array([84., 12., 28., 21.])
```